

Documentación de la aplicación (Flask + HTML/CSS)

1. Objetivo y alcance

Esta aplicación implementa un panel para un asistente personal basado en lenguaje natural. El foco de esta documentación es explicar, con rigor académico, cómo está organizada y cómo funciona la capa web con **Flask** y **Jinja2**, partiendo de que el alumnado ya conoce HTML y CSS, pero **no** Flask.

El proyecto está dividido en:

- **Rutas Flask** que reciben las peticiones HTTP y deciden qué plantilla renderizar.
 - **Plantillas Jinja2** que estructuran la interfaz en un **layout base** y **parciales** (panel izquierdo, cabecera, panel central, panel derecho y pie).
 - **Recursos estáticos** (CSS/JS) servidos por Flask desde la carpeta `static/`.
-

2. Requisitos y ejecución

2.1 Dependencias

- Python 3.10+
- Flask 3.0.0

2.2 Puesta en marcha (desarrollo)

```
python3 -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -r requirements.txt
python app.py                  # o: flask --app app run --debug
```

La aplicación expone por defecto `http://127.0.0.1:5000/`.

3. Estructura del proyecto

```
GoalMind-AI/
  app.py                # Aplicación Flask con rutas básicas
  requirements.txt
  README.md
  static/
    css/styles.css      # Hoja de estilos principal
    js/app.js           # Script base para futuras funcionalidades
  templates/
```

<code>base.html</code>	# Layout principal (herencia Jinja)
<code>dashboard.html</code>	# Página que compone panel central y derecho
<code>partials/</code>	
<code>left_panel.html</code>	# Panel izquierdo (navegación y estado)
<code>header.html</code>	# Cabecera superior
<code>center_panel.html</code>	# Panel central (calendario)
<code>right_panel.html</code>	# Panel derecho (objetivos, tareas, etc.)
<code>footer.html</code>	# Pie de página

- `app.py` concentra las rutas mínimas. En proyectos grandes conviene migrar a **Blueprints** (véase §10).
- `templates/base.html` define la estructura marco y los puntos de extensión (`{% block content %}`).
- Los **parciales** encapsulan cada zona del diseño para favorecer la mantenibilidad y la reutilización.
- `static/` contiene los recursos cacheables del navegador. Flask los sirve mediante `url_for('static', ...)`.

4. Anatomía de una petición en Flask

1. El navegador solicita una URL (por ejemplo, `/objetivos`).
2. Flask resuelve la ruta en `app.py` y ejecuta la función Python asociada.
3. La función llama a `render_template(...)`, opcionalmente pasando **variables de contexto** (por ejemplo, `page="objetivos"`).
4. Jinja2 procesa la plantilla solicitada:
 - Extiende de `base.html` (herencia).
 - Inserta los **parciales** necesarios (`{% include ... %}`).
 - Sustituye variables (`{{ ... }}`) y evalúa estructuras de control (`{% if ... %}`, `{% for ... %}`).
5. Se devuelve HTML final al navegador.

Este **ciclo request–response** separa claramente:

- **Controlador** (función Python de la ruta) y
 - **Vista** (plantillas Jinja2).
-

5. Rutas y controladores (`app.py`)

En esta demo, las rutas son mínimas y devuelven la misma vista base con un indicador de página:

```

from flask import Flask, render_template

app = Flask(__name__)

@app.route("/")
def dashboard():
    return render_template("dashboard.html", page="dashboard")

@app.route("/agenda")
def agenda():
    return render_template("dashboard.html", page="agenda")

# ... objetivos, tareas, estadísticas, config

```

Puntos clave:

- **render_template** busca archivos en `templates/`.
- La variable `page` permite marcar en el menú la sección activa y ajustar el encabezado de la página sin replicar plantilla.
- Esta aproximación minimiza duplicación; para contenido muy distinto, se recomienda una plantilla por sección.

6. Plantillas Jinja2: herencia, bloques e inclusiones

6.1 Layout base (`templates/base.html`)

Define el armazón de la interfaz: panel izquierdo fijo y área principal con cabecera, contenido y pie. Elementos relevantes:

- Carga del CSS con `{{ url_for('static', filename='css/styles.css') }}`.
- Inclusión del panel izquierdo: `{% include "partials/left_panel.html" %}`.
- **Bloque de contenido:**
`{% block content %}{% endblock %}`
 Las vistas hijas lo rellenan.
- Inclusión de cabecera y pie en `main`.

6.2 Vista hija (`templates/dashboard.html`)

Extiende el layout y compone los paneles central y derecho:

```

{% extends "base.html" %}
{% block content %}

```

```

<div class="grid">
    {% include "partials/center_panel.html" %}
    {% include "partials/right_panel.html" %}
</div>
{% endblock %}

```

6.3 Parciales

- `left_panel.html`: navegación y estado del sistema. Usa `url_for` para enlaces estables y `aria-current` condicionado por `page`.
- `header.html`: ruta actual y acciones rápidas.
- `center_panel.html`: calendario semanal estructurado como tabla accesible. La versión demo es estática; en producción se pobla con datos.
- `right_panel.html`: objetivos, tareas, alarmas y formularios de ejemplo.
- `footer.html`: pie informativo.

6.4 Variables de contexto y control de flujo

El patrón habitual es pasar datos del controlador a la plantilla:

```

eventos = [
    {"dia": "Mie", "inicio": "08:15", "fin": "08:45", "titulo": "Revisión objetivos"}
]
return render_template("dashboard.html", page="agenda", eventos=eventos)

```

Y recorrer en Jinja:

```

{% for ev in eventos %}
    <!-- pintar el evento en su celda -->
{% endfor %}

```

7. Gestión de recursos estáticos

- Flask expone `static/` por defecto en `/static/...`
- La **forma correcta** de referenciar un recurso es `{{ url_for('static', filename='css/styles.css') }}`. Esto evita rutas rotas y habilita cache busting cuando se cambian archivos.
- El CSS define variables de tema, rejillas y estilos para las tarjetas y el calendario.
- `static/js/app.js` es un punto de entrada previsto para interacciones (por ejemplo, arrastrar y soltar en el calendario o llamadas AJAX).

8. Accesibilidad y responsividad

- Se emplean roles y atributos ARIA esenciales en navegación y tablas.

- El calendario utiliza cabeceras pegajosas para legibilidad.
- Existe una clase utilitaria `visually-hidden` para contenido descriptivo no visual.
- El diseño se adapta con media queries: en pantallas estrechas, el sidebar pasa a disposición superior y los paneles se apilan.

9. Extender la aplicación: añadir una nueva página

1. Ruta en `app.py`:

```
@app.route("/proyectos")
def proyectos():
    datos = cargar_proyectos() # hipotético
    return render_template("proyectos.html", page="proyectos", proyectos=datos)
```

2. Plantilla `templates/proyectos.html`:

```
{% extends "base.html" %}
{% block content %}
<section class="card">
    <h2>Proyectos</h2>
    <ul>
        {% for p in proyectos %}
        <li>{{ p.nombre }} - {{ p.estado }}</li>
        {% endfor %}
    </ul>
</section>
{% endblock %}
```

3. Navegación en `partials/left_panel.html` añadiendo un `<li>` con `aria-current` condicionado por `page`.

10. Buenas prácticas de arquitectura en Flask

En proyectos docentes de tamaño medio o grande se recomienda:

10.1 Blueprints y factoría de aplicaciones

- **Blueprints** permiten modularizar rutas por dominio (por ejemplo, `dashboard`, `agenda`, `auth`).
- **App Factory** facilita configurar la aplicación por entorno (desarrollo, pruebas, producción).

Esquema sugerido:

`app/`

```

__init__.py          # create_app(): registra blueprints, config, extensiones
dashboard/
    __init__.py      # blueprint dashboard_bp
    routes.py
agenda/
    __init__.py
    routes.py
templates/...
static/...

__init__.py (simplificado):

from flask import Flask
from .dashboard import dashboard_bp
from .agenda import agenda_bp

def create_app():
    app = Flask(__name__)
    app.config.from_mapping(SECRET_KEY="cambiar")
    app.register_blueprint(dashboard_bp)
    app.register_blueprint(agenda_bp, url_prefix="/agenda")
    return app

```

10.2 Separación de capas

- **Rutas:** controlan el flujo y validan entrada.
- **Servicios:** encapsulan lógica de negocio (por ejemplo, sincronización de calendario).
- **Persistencia:** aislada y testeable (DAO o repositorios).

11. Plantillas: recomendaciones didácticas

- **Herencia en cascada:** un layout base y, si es necesario, sublayouts por sección para no sobrecargar `base.html`.
- **Macros de Jinja** para componentes repetidos (tarjetas, barras de progreso, etiquetas de estado).
- **Filtro `url_for`** siempre para enlaces internos y estáticos.
- **Evitar lógica compleja** en la plantilla; la vista debe recibir datos ya preparados.

Ejemplo de macro (`templates/macros/components.html`):

```

{% macro kpi(label, valor, porcentaje) -%}
<div class="kpi">
    <div class="muted">{{ label }}</div>
    <div class="val">{{ valor }}</div>
    <div class="bar"><span style="width:{{ porcentaje }}%"></span></div>

```

```
</div>
{%%- endmacro %}
```

Uso:

```
{% from "macros/components.html" import kpi %}
<section class="card">
  <h2>Estadísticas</h2>
  <div class="stats">
    {{ kpi("Tareas completadas (7 días)", 18, 72) }}
    {{ kpi("Objetivos cumplidos (30 días)", 4, 40) }}
  </div>
</section>
```

12. Formularios y procesamiento de datos

El panel derecho incluye un formulario de ejemplo no funcional. Para hacerlo operativo:

1. Definir la ruta POST con validación:

```
from flask import request, redirect, url_for, flash

@app.route("/tareas/nueva", methods=["POST"])
def crear_tarea():
    titulo = request.form.get("titulo", "").strip()
    if not titulo:
        flash("El título es obligatorio", "error")
        return redirect(url_for("tareas"))
    # Persistir en base de datos...
    flash("Tarea creada correctamente", "ok")
    return redirect(url_for("tareas"))
```

2. En la plantilla, usar `<form method="post" action="{{ url_for('crear_tarea') }}">...</form>`.

Para producción se recomienda integrar **CSRF** (por ejemplo, con Flask-WTF).

13. Pruebas y calidad

- **Pruebas de rutas** con el cliente de test de Flask (`app.test_client()`).
- **Validación de HTML** con herramientas de linting o validadores.
- **Accesibilidad**: verificación manual con lectores de pantalla y herramientas de auditoría.

Ejemplo con `pytest`:

```
def test_dashboard(client):  
    resp = client.get("/")  
    assert resp.status_code == 200  
    assert b"Asistente Personal LLM" in resp.data
```
